Hochschule Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

Polling & Web Sockets

Web Engineering 1 (319300)

Vorlesung 04

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-11-03

Agenda

- Polling & Long Polling
- Web Sockets

Polling & Long Polling

Das Problem

- HTTP ist ein **Request-Response-Protokoll**: Der Client fragt, der Server antwortet
- Der Server kann von sich aus **nichts** an den Browser schicken
- Frage: Wie erfährt die Seite von neuen Daten (Chatnachricht, Benachrichtigung, Auftragsstatus)?

Drei Ansätze mit steigender Komplexität:

Ansatz	Idee
Polling	Client fragt in festen Abständen nach
Long Polling	Server hält die Anfrage offen, bis es etwas Neues gibt
Server-Sent Events, Web Sockets	Dauerhafte Verbindung (kommt später)

Polling

- Der Client sendet in regelmäßigen Abständen eine Anfrage, egal ob es etwas Neues gibt
- Einfachste Variante
- Funktioniert mit jedem Server und jedem Proxy

Polling

Beispiel

JavaScript

```
async function pollStatus() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  }
}
setInterval(pollStatus, 5000); // alle 5 Sekunden
```

Polling Umsetzung

Problem von `setInterval`: Dauert eine Antwort länger als das Intervall, überlappen sich beide Anfragen.

Besser: rekursives `setTimeout`. Die nächste Anfrage startet erst, wenn die letzte fertig ist.

Weitere Verbesserungen:

- Polling stoppen, wenn der Tab im Hintergrund ist (`document.visibilityState`, Event `visibilitychange`)
- Bei Fehlern das Intervall vergrößern (Exponential Backoff: 5s, 10s, 20s, …)
- Abbruch über `AbortController`, z.B. beim Verlassen der Seite
- Serverseitig `ETag`/`If-None-Match` nutzen: Antwort `304 Not Modified` spart Daten

Bewertung

Vorteile:

- Sehr einfach zu implementieren
- Funktioniert überall (Proxys, Firewalls, Caches)
- Server bleibt zustandslos

Nachteile

- Latenz bis zu einem Intervall (bei 5s im Schnitt 2.5s Verzögerung)
- Viele unnötige Requests, wenn sich selten etwas ändert
- Kurzes Intervall = Extra Last auf Server und Netz
- Langes Intervall = Träge UI

Typischer Einsatz: Dashboards, Statusabfragen von lang laufenden Jobs, wenn Verzögerungen von Sekunden egal sind

Long Polling

Long Polling

Idee: Der Server beantwortet die Anfrage nicht sofort, sondern hält sie offen, bis es etwas Neues gibt (oder ein Timeout erreicht ist). Danach öffnet der Client sofort die nächste Anfrage.

Ablauf

1. Client sendet Anfrage
2. Server wartet (z.B. bis zu 30s)
3. Neue Daten da: Server antwortet sofort. Timeout: Server antwortet leer (z.B. 204 No Content)
4. Client verarbeitet die Antwort und startet direkt die nächste Anfrage

Long Polling bewertet

Vorteile:

- Fast sofortige Zustellung
- Weniger leere Requests als Polling
- Läuft über normales HTTP

Nachteile:

- Der Server hält viele offene Verbindungen
- Proxys und Load Balancer beenden lange Verbindungen (Timeout oft 30 bis 60s)
- Pro Domain gibt es Verbindungslimits (HTTP/1.1: meist 6 pro Browser)
- Zwischen zwei Anfragen entsteht kurz eine „Lücke“

Long Polling bewertet

- Server-Timeout kürzer als der Proxy-Timeout wählen
- Server muss viele wartende Anfragen effizient halten können (asynchrones/nicht blockierendes I/O, bei Thread-pro-Request-Modellen wird das sehr schnell eng)
- Bei Fehlern immer mit Pause und Backoff neu verbinden, sonst entsteht eine „Thundering Herd“, wenn der Server wieder erreichbar ist

Vergleich und Ausblick

	Polling	Long Poll.	SSE	WebSocket
Richtung	Client → Server	Client → Server	Server → Client	beidseitig
Latenz	Intervall	gering	gering	gering
Verbindung	neu pro Anfrage	neu pro Antwort	dauerhaft	dauerhaft
Protokoll	HTTP	HTTP	HTTP (`EventSource`)	eigenes (`ws://`, `wss://`)
Aufwand	sehr gering	mittel	gering	höher

Vergleich und Ausblick

Faustregel:

- Selten ändernde Daten, Sekunden egal: **Polling**
- Neuigkeiten sollen schnell ankommen, nur Server → Client: **SSE** (oder Long Polling als Fallback)
- Echte Zwei-Wege-Kommunikation (Chat, Spiele, Kollaboration): **WebSocket**

Kurzer Ausblick auf SSE:

```javascript
const source = new EventSource("/api/events");
source.onmessage = (event) =>
console.log(JSON.parse(event.data));
```

Typische Fehler

- `setInterval` mit `async`-Funktion: Anfragen überlappen sich, wenn der Server langsam ist
- **Kein Abbruch**: Polling läuft weiter, obwohl die Seite oder Komponente längst weg ist (Memory Leak, vgl. Folie zu Garbage Collection)
- **Zu kurze Intervalle**: 1-Sekunden-Polling von 1000 Nutzern sind 1000 Requests pro Sekunde
- **Endlosschleife ohne Pause bei Fehlern**: Der Client hämmert dann den Server mit Fehlversuchen zu

Probleme

Ausgangslage:

- HTTP hat eine feste Rollenverteilung: **Nur der Client darf fragen**, der Server kann nur antworten
- Bei klassischen Seiten egal
- Bei Echtzeitanwendungen ein wirkliches Problem, z.B. Chat, Live-Ticker oder Multiplayer-Spiele (Server soll Client auch erreichen)

Vier Stufen der Verbindung:

1. **Kurzes Polling:** Client fragt alle paar Sekunden. Verschwendung und Latenz
2. **Long Polling:** Server hält Antwort offen, bis ein Ergebnis eintrifft. Jede Nachricht ist immer noch eine volle Anfrage
3. **SSE:** Dauerhafte offene HTTP-Verbindung für Server → Client mit eingebautem Reconnect. Client → Server bleibt Request
4. **WebSocket:** ?

Web Sockets

Welches Problem wird gelöst?

Welches Problem wird gelöst?

- Klassisches HTTP ist ein Request-Response-Protokoll
- Nur der Client intialisiert Kommunikation
- Für real-time Anwendungen (Chats, Live Dashboards, Multiplayer Spiele, ...) würde sonst Polling benötigt werden
- Wiederholte HTTP Requests wären verschwenderisch und *laggy*

Technische Übersicht

Welches Problem wird gelöst?

- Klassisches HTTP ist ein Request-Response-Protokoll
- Nur der Client intialisiert Kommunikation
- Für real-time Anwendungen (Chats, Live Dashboards, Multiplayer Spiele, …) würde sonst Polling benötigt werden
- Wiederholte HTTP Requests wären verschwenderisch und *laggy*

Was ist eine WebSocket?

Technische Übersicht

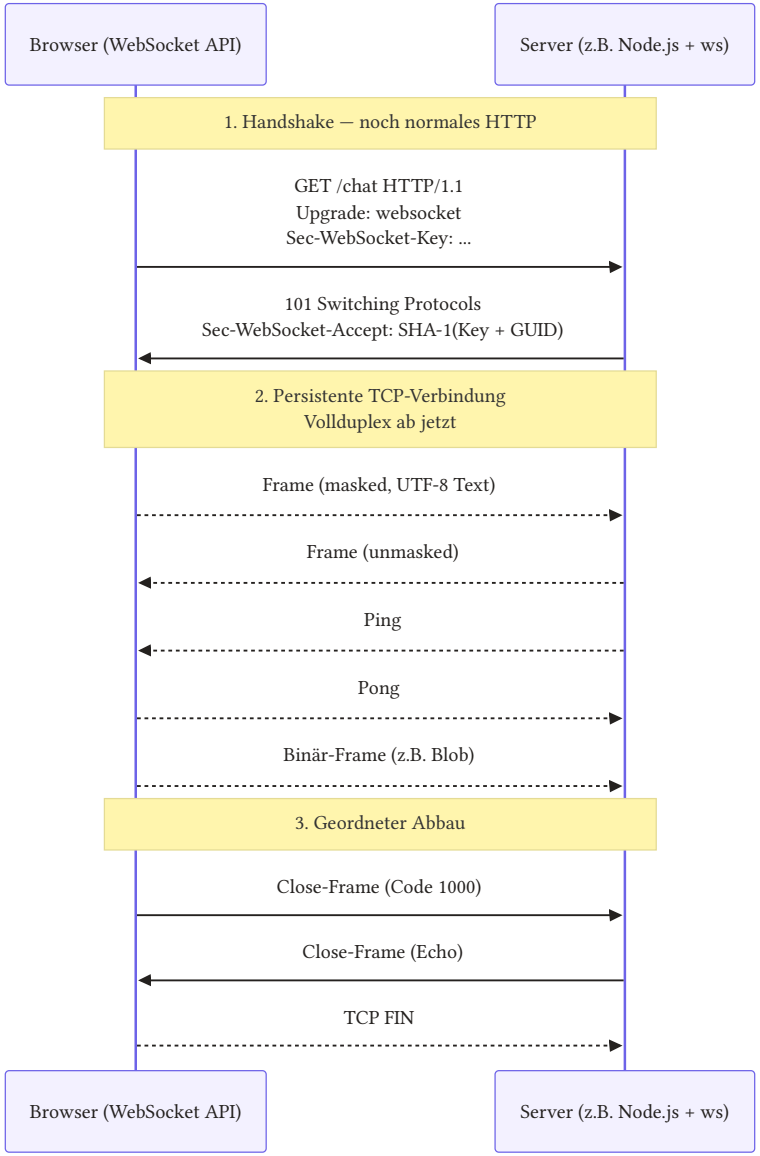
Welches Problem wird gelöst?

- Klassisches HTTP ist ein Request-Response-Protokoll
- Nur der Client intialisiert Kommunikation
- Für real-time Anwendungen (Chats, Live Dashboards, Multiplayer Spiele, ...) würde sonst Polling benötigt werden
- Wiederholte HTTP Requests wären verschwenderisch und *laggy*

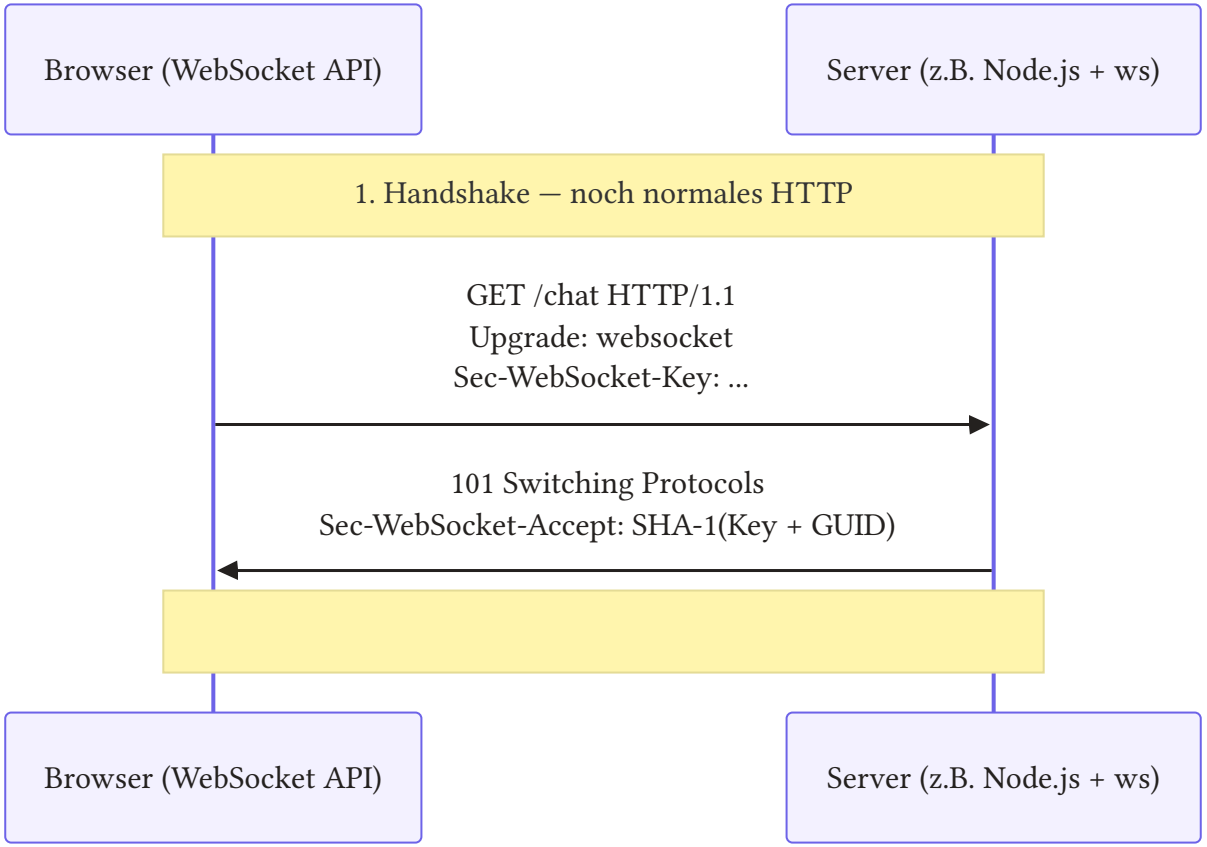
Was ist eine WebSocket?

- Eine einzelne persistente TCP Verbindung
- Wird über HTTP ausgehandelt (Handshake)
- Wird danach zu einer full-duplex bzw. einem bidirektionalem Kanal (definiert nach RFC 6455)
- Beide Seiten können Nachrichten schicken mit minimalem Overhead

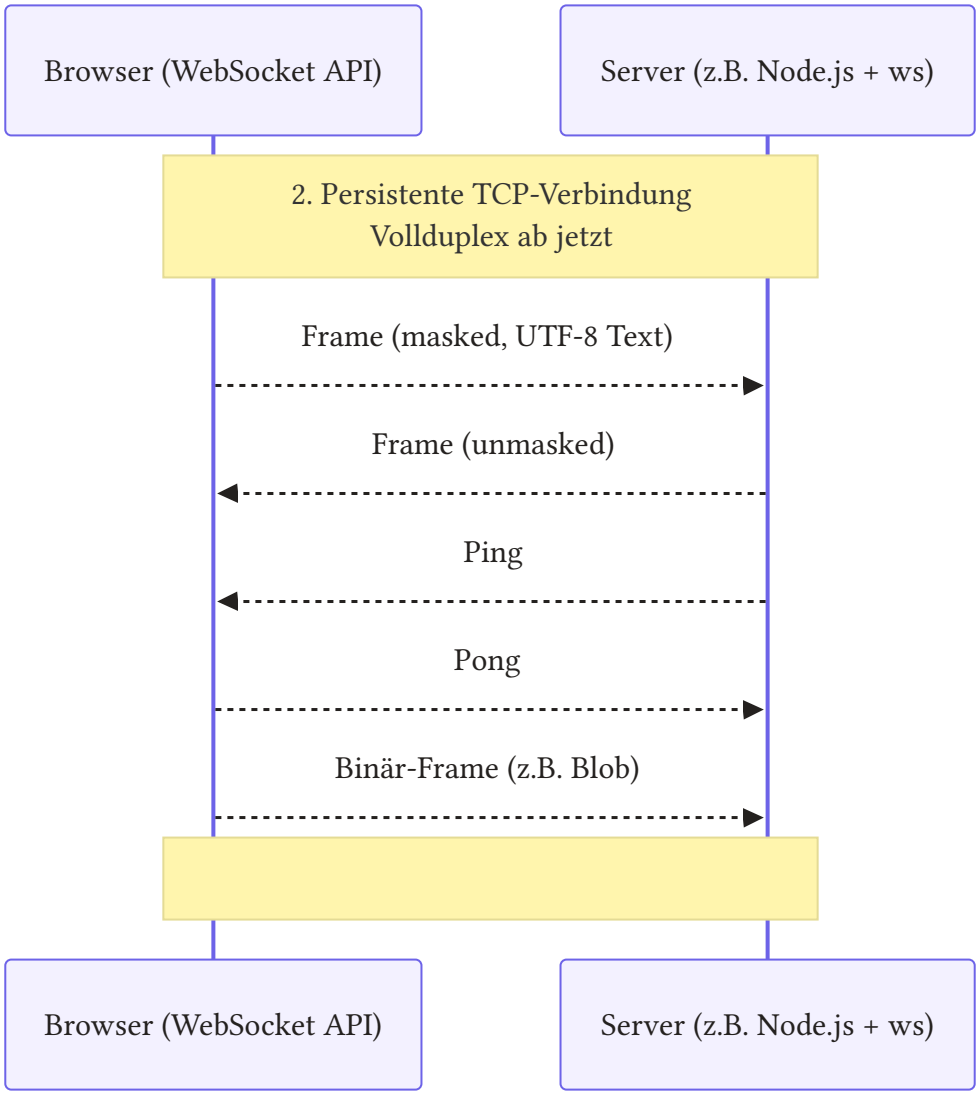
Ablauf


Browser (WebSocket API)
Server (z.B. Node.js + ws)
1. Handshake — noch normales HTTP
GET /chat HTTP/1.1
Upgrade: websocket
Sec-WebSocket-Key: ...
101 Switching Protocols
Sec-WebSocket-Accept: SHA-1(Key + GUID)
2. Persistente TCP-Verbindung
Vollduplex ab jetzt
Frame (masked, UTF-8 Text)
Frame (unmasked)
Ping
Pong
Binär-Frame (z.B. Blob)
3. Geordneter Abbau
Close-Frame (Code 1000)
Close-Frame (Echo)
TCP FIN
Browser (WebSocket API)
Server (z.B. Node.js + ws)

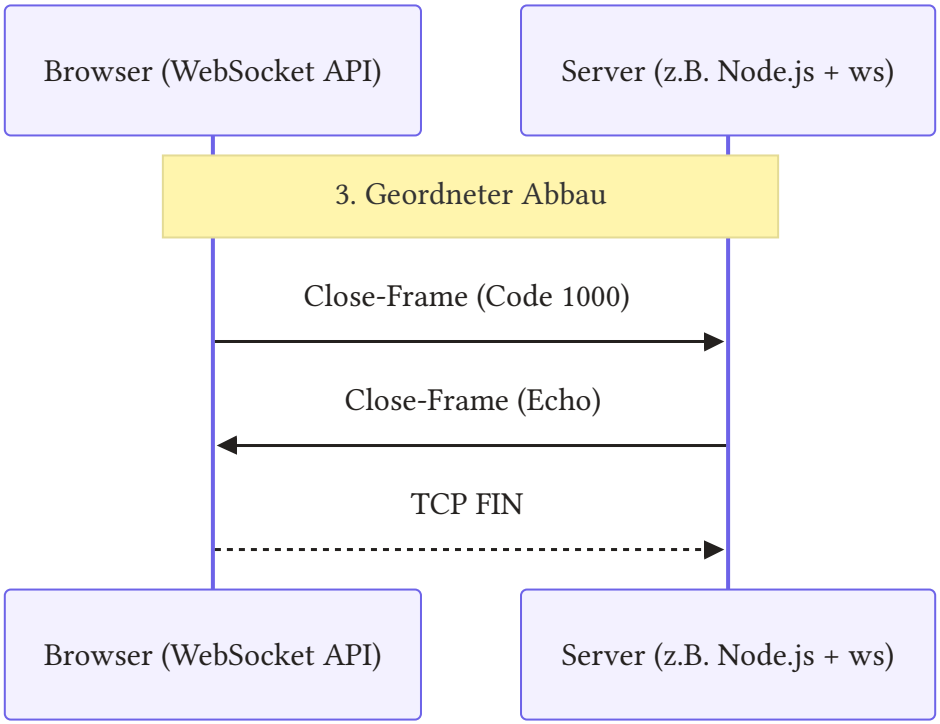
Ablauf - Phase 1



Ablauf - Phase 2



Ablauf - Phase 3



HTTP vs. Web Sockets

HTTP

- Halbduplex: pro Kommunikation genau eine Richtung → erst fragen, dann antworten
- Nur der Client kann initiieren; der Server schweigt, bis Request kommt
- Jede Nachricht kostet einen vollen Request-Zyklus (500 Byte Header)
- Für Neuigkeiten: Polling → wiederholte, meist leere Anfrage

Web Sockets

- Eine einzige persistente Verbindung für die gesamte Sitzung
- Beide Seiten senden jederzeit, ohne neuen Verbindungsaufbau
- Jede Nachricht ist ein schlanker Frame (maximal 14 Byte Overhead)
- Der Server pusht aktiv: keine Anfrage, keine Wartezeit

Der Handshake

Die Verbindung startet mit einer normalen HTTP GET Request mit einem `Upgrade` Header:

```
1   GET /chat HTTP/1.1
2   Host: exmaple.com
3   Upgrade: websocket
4   Connection: Upgrade
5   Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
6   Sec-WebSocket-Version: 13
```

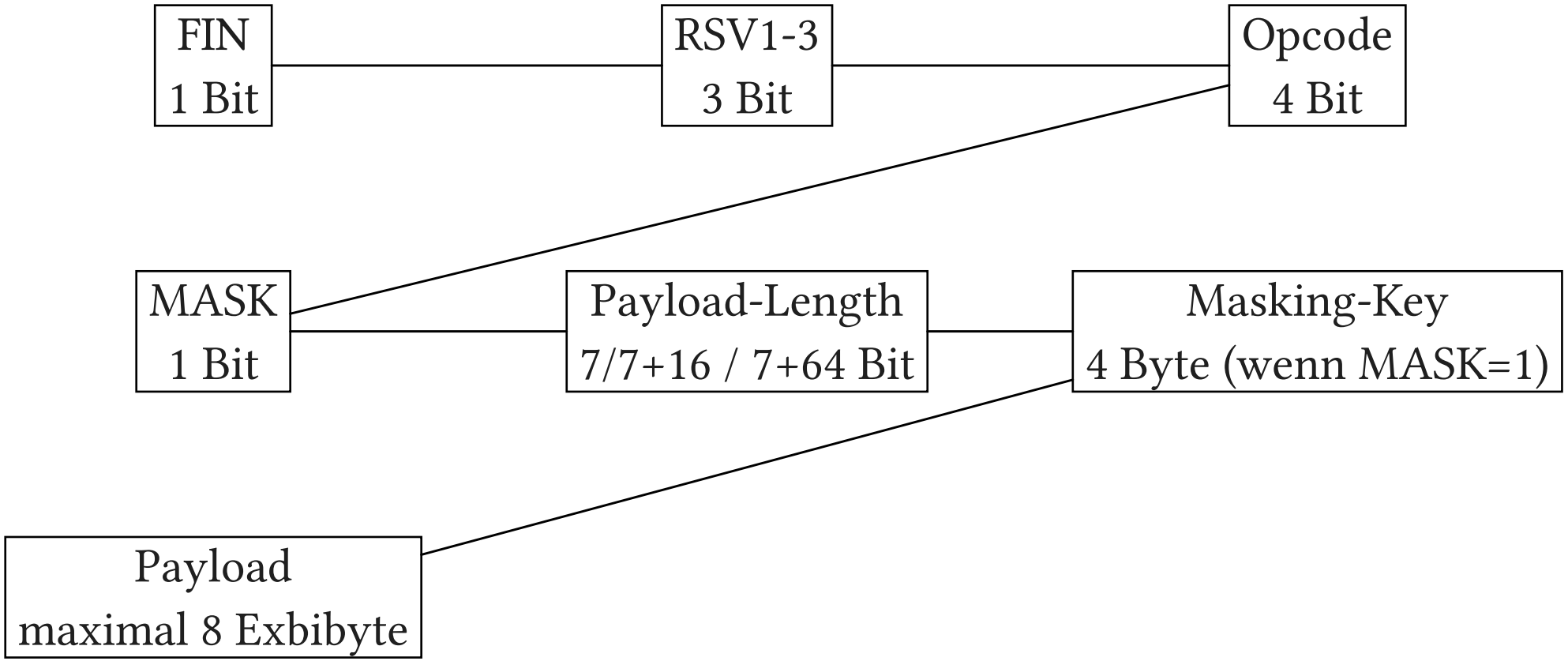
- Der Server antwortet mit `101 Switching Protocols` und einem `Sec-WebSocket-Accept` Value
- Beweis dafür, dass der Server WebSockets benutzen kann
- HTTP ist jetzt fertig → TCP übernimmt mit dem **WebSocket framing protocol**

Protokoll Komponenten

- **Frames:**
  - Client → Server Frames müssen maskiert werden (4-byte XOR Key als Sicherheitsmaßnahme)
  - Server → Client Frames werden nicht maskiert
- **Frame types:** Text (UTF-8), Binary, Ping/Pong (keep-alive), Close
- **Fragmentation:** Große Nachrichten können in mehrere Frames aufgeteilt werden
- **Closing:** Handshake um die Verbindung ordentlich zu schließen (close frame + TCP FIN), Verbindung wird sonst einfach abgebrochen

Aufbau eines Frames

- Jede Nachricht nach dem Upgrade ist ein **Frame**
- Frame-Format ist binär definiert: Header ist zwischen 2 und 14 Byte groß



Komponenten

FIN (1 Bit): Endfragment-Flag:

- `1` = die Nachricht ist vollständig in diesem Frame
- `0` = es folgen Kontinuations-Frames (Opcode `0`) bis ein Frame mit `FIN=1` kommt

RSV1-RSV3 (je 1 Bit): reserviert, müssen `0` sein, **außer** eine Erweiterung ist ausgehandelt

Opcode (4 Bit): Typ des Frames

Opcode	Frame	Bedeutung
`0x0`	Continuation	Fortsetzung einer fragmentierten Nachricht
`0x1`	Text	UTF-8-Nachricht, als Grenzes dekodierbar
`0x2`	Binary	beliebige Binärdaten (ArrayBuffer/Blob)
`0x8`	Close	leitet den geordneten Abbau ein
`0x9`	Ping	Herzschlag-Anfrage
`0xA`	Pong	Herzschlag-Antwort
`0x3-0x7, 0xB-0xF`	reserviert	ungültig im normalen Betrieb

Komponenten

MASK (1 Bit): muss bei allen Client-Frames `1` sein und `0` bei Server-Frames. Ein Server, der einen unmaskierten Client-Frame erhält (oder umgekehrt maskierte vom Server), trennt die Verbindung (Protokollfehler 1002).

Payload-Länge (7 Bit + optional): drei Stufen:

1. Länge `0-125`: passt direkt in die 7 Bit, kein Zusatzfeld
2. Wert `126`: die nächsten **2 Byte** (16 Bit, bis 65.535) tragen die echte Länge
3. Wert `127`: die nächsten **8 Byte** (64 Bit) tragen die echte Länge

Komponenten

Masking-Key (4 Byte, bei MASK=1): ein zufälliger XOR-Schlüssel

- Payload wird byteweise mit `payload[i] XOR key[i mod 4]` maskiert
- Empfänger (Server) demaskiert identisch
- **Beachte:** Der Frame-Header wird nie selbst maskiert
- Maskierung basiert auf der Regel: `a XOR k XOR k = a`
- **Ziel:** Proxy soll Bytes nicht als HTTP wahrnehmen
- **Risiko:** Requests in den Proxy schmuggeln um andere Daten zu erhalten

Payload: die eigentlichen Daten, 0 Byte bis zur 64-Bit-Grenze.

Konkretes Beispiel (Server → Client)

```
1    81 05 48 61 6C 6C 6F
```

Byte(s)	Hex	Binär	Bedeutung
1	`81`	`1000 0001`	FIN=1 (eine komplette Nachricht), RSV=000, Opcode 0x1 = Text-Frame
2	`05`	`0000 0101`	MASK=0 (Server sendet nie maskiert), Payload-Länge = 5
3-7	`48 61 6C 6C 6F`		Payload: „Hallo“ in UTF-8

Konkretes Beispiel (Server → Client)

Im Browser:

- WebSocket-API des Browsers liest den Frame-Header
- Erkennt Opcode `0x1` (Text)
- Dekodiert den Payload als UTF-8
- Übergibt Ergebnis dem Handler

```javascript
socket.onmessage = (event) => {
  event.data; // "Hallo"
  event.origin; // "wss://example.com"
  event.lastEventId; // "" (nur bei SSE relevant)
  event.ports; // []
}
```

Beispiel für mehrere Frames (Fragmentierung)

Frame 1	`FIN=0, Opcode=0x1, Payload="Web "`	`01 04 57 65 62 20`
Frame 2	`FIN=1, Opcode=0x0, Payload="Engineering"`	`80 0B 45 6E …`

- `MASK=0` wurde gesetzt, also Server → Client
- Erst mit dem zweiten Frame ist die Nachricht vollständig
- Wichtig: Der Opcode (`0x1` Text) steht nur im ersten Frame
- Alle Continuation-Frames tragen nur `0x0`
- Ein anderer Opcode im Continuation-Frame wäre Protokollverstoß

Beispiel für mehrere Frames (Fragmentierung)

Frame 1

Hex

```
01 04 57 65 62 20
```

Byte 1 in Binär

```
  0000 0001
  ||-| |--| Opcode = 0x1 -> Text
  || RSV = 000 -> Keine Erweiterung
  | FIN = 0 -> Nachricht noch nicht zu Ende
```

Beispiel für mehrere Frames (Fragmentierung)

Frame 1

Hex

```
1  01 04 57 65 62 20
```

Byte 2 in Binär

```
1    0000 0100
2    |--| |--| Payload-Länge = 4 -> Payload besteht aus 4 Bytes
3    |MASK = 0 -> kein Masking-Key, kein XOR (Server ist Absender)
```

Beispiel für mehrere Frames (Fragmentierung)

Frame 1

	Hex
1	01 04 57 65 62 20

- `0101 0111` → `W`
- `0110 0101` → `e`
- `0110 0010` → `b`
- `0010 0000` → ` `

Server & Client API

```JavaScript
const socket = new WebSocket(
  "wss://example.com/chat"
);
socket.onopen = () => socket.send("Hello");
socket.onmessage = (event) => console.log(event.data);
socket.onclose = (event) => console.log(event.code, event.reason);
socket.onerror = (e) => console.error(e);
```

```JavaScript
import { WebSocketServer } from "ws";
const wss = new WebSocketServer({ port: 8080 });

wss.on("connection", (ws) => {
  ws.on("message", (data) => wss.clients.forEach(c =>
  c.send(data)));
});
```

Verbindungen zu vorherigen Themen

Event Loop: `onmessage` Callbacks sind normale Tasks, Nachrichten die ankommen werden in die Queue gelegt und asynchron abgearbeitet (wie `setTimeout`/`Promise`)

Promises/async-await: Connection, Reconnection und send ackowledgements werden in den meisten Bibliotheken als Promises verpackt (auf Browser Ebene ereignisbasiert `onopen`/`onmessage`/`onclose`)

Vergleich zu Alternativen

	Polling	Long Polling	SSE	WebSockets
Richtung	Client → Server	Client → Server	Server → Client	**Bidirektional**
Latenz	Hoch	Mittel	Niedrig	**geringste**
Overhead	komplette HTTP Header	komplette HTTP Header	wenig	max. 14 Byte
Protokoll	HTTP	HTTP	HTTP	eigenes Protokoll

Fun-Fact: Server-Sent Events (SSE) wurden wieder populär für AI Streaming (z.B. ChatGPT Token Streams)

Zusammenfassung

Zusammenfassung

Polling:
- Der Client fragt in festen Intervallen an, ob neue Daten vorliegen
- Meist kommt eine leere Antwort (Bringt Latenz und Verschwendung von Ressourcen)
- Jede Request hat volle HTTP-Header

Long Polling:
- Client fragt an, der Server hält die Antwort offen bis ein Ergebnis vorliegt
- Leerfahrten entfallen
- Jede Nachricht kostet trotzdem einen kompletten HTTP-Zyklus

Server-Sent Events (SSE):
- Dauerhaft offene HTTP Verbindung
- Server streamt Daten zum Client
- Nur Server $\rightarrow$ Client, Client muss weiterhin Requests stellen

WebSocket:
- Nach einmaligem HTTP-Upgrade besteht dauerhafte vollduplex Verbindung
- Client und Server senden jederzeit Nachrichten ohne neuen Verbindungsaufbau